App.jsx code explained:

1. IMPORTS

```
import { useState } from "react";
import "./App.css";
```

What this does:

- `useState` → React hook for managing data (state)
- `"./App.css"` → imports our styling (the file we explained earlier)

This connects:

- Logic (JS)
- UI (CSS)

2. COMPONENT START

```
function App() {
```

This is our **main React component** (our app UI + logic)

3. STATE VARIABLES (VERY IMPORTANT)

```
const [image, setImage] = useState(null);
const [preview, setPreview] = useState(null);
const [uploadedImageUrl, setUploadedImageUrl] = useState(null);
```

Purpose:

State	Meaning
image	actual uploaded file
preview	local preview before upload
uploadedImageUrl	image stored in S3

```
const [firstName, setFirstName] = useState("");
const [lastName, setLastName] = useState("");
```

Stores user input (form fields)

```
const [message, setMessage] = useState("Please enter details and upload an
image.");
const [loading, setLoading] = useState(false);
```

State	Purpose
message	feedback to user
loading	shows upload in progress

4. MAIN FUNCTION: sendImage()

```
async function sendImage(e) {
```

This runs when user clicks **Upload**

4.1 PREVENT PAGE RELOAD

```
e.preventDefault();
```

Stops form from refreshing page

4.2 VALIDATION

```
if (!image || !firstName || !lastName) {
```

Ensures:

- Image uploaded
- Name entered

✓ Good UX practice

5. READ IMAGE FILE

```
const reader = new FileReader();
```

Reads image from user's computer

5.1 AFTER FILE IS READ

```
reader.onloadend = async () => {
```

Runs after image is converted

5.2 CONVERT TO BASE64

```
const base64Image = reader.result.split(",")[1];
```

Converts image into:

```
base64 string
```

✓ Required for API (Lambda expects base64)

6. API CALL (CORE CLOUD PART)

```
const response = await fetch(
  "https://h3v6z64lxd.execute-api.eu-west-1.amazonaws.com/dev/upload",
```

This calls our:

- API Gateway → Lambda → Backend

REQUEST DETAILS

```
method: "POST",
headers: {
  "Content-Type": "application/json",
},
```

Sending JSON data

BODY SENT TO LAMBDA

```
body: JSON.stringify({
  firstName,
  lastName,
  image: base64Image,
}),
```

This is what backend receives:

```
{
  "firstName": "Larry",
  "lastName": "Page",
  "image": "base64..."
}
```

7. LOADING STATE

```
setLoading(true);
setMessage("Uploading...");
```

UI shows:

```
Uploading...
```

8. GET RESPONSE

```
const data = await response.json();
```

Receives response from Lambda

9. HANDLE RESPONSE

SUCCESS

```
if (data.Message === "Employee registered successfully")
```

Shows:

```
setMessage(`✅ ${firstName} ${lastName} registered successfully`);
```

BUILD S3 IMAGE URL

```
const s3Url = `https://samuel-visitor-images.s3.eu-west-
1.amazonaws.com/${data.ImageKey}`;
```

Converts:

```
ImageKey → full S3 URL
```

✓ Allows image display from cloud

FAILURE

```
else if (data === "Failure" || data.Message === "Failure")
```

Shows:

```
No face detected or authentication failed
```

OTHER ERRORS

```
setMessage("❌ " + JSON.stringify(data));
```

Displays raw backend error

10. ERROR HANDLING

```
catch (error) {
  console.error(error);
  setMessage("❌ Error connecting to API");
}
```

If API fails:

- Shows error message
- Logs issue

11. FINALLY (CLEANUP)

```
finally {
  setLoading(false);
}
```

Stops loading state

12. START FILE READING

```
reader.readAsDataURL(image);
```

Converts file → base64

13. UI RENDER (RETURN BLOCK)

APP TITLE

```
<h2>Facial Recognition System</h2>
```

FORM

```
<form onSubmit={sendImage}>
```

Submits to our function

INPUT FIELDS

```
<input type="text" placeholder="First Name" />
<input type="text" placeholder="Last Name" />
```

Controlled inputs using state

IMAGE UPLOAD

```
<input type="file" accept="image/*" />
```

PREVIEW IMAGE

```
setPreview(URL.createObjectURL(file));
```

Shows image BEFORE upload

BUTTON

```
<button disabled={loading}>
```

Disabled while uploading

14. MESSAGE DISPLAY

```
<strong>{message}</strong>
```

Shows:

- success
- failure

- loading

15. LOCAL PREVIEW

```
{preview && <img src={preview} />}
```

Shows uploaded image locally

16. S3 IMAGE DISPLAY

```
{uploadedImageUrl && <img src={uploadedImageUrl} />}
```

Shows cloud-stored image

FULL FLOW (END-TO-END)

```
User selects image
      ↓
Preview shown
      ↓
Click Upload
      ↓
Convert to base64
      ↓
Send to API Gateway
      ↓
Lambda processes
      ↓
Response returned
      ↓
Show success/failure
      ↓
Display S3 image
```

SUMMARY

This file handles:

- User input
- Image processing

- API communication
- UI feedback
- Cloud integration

“This React component manages user input, converts uploaded images to base64, sends them to an AWS API Gateway endpoint for processing, and dynamically updates the UI based on the response. It also provides real-time feedback and displays both local previews and cloud-stored images.”